SOLUTIONS

Question 1 – Solution

Fill the 3-gallon jug and pour it into the 4-gallon jug. Fill the 3-gallon jug again and pour it into the 4-gallon jug until the 4-gallon jug becomes full, leaving 2 gallons in the 3-gallon jug. Empty the 4-gallon jug and transfer the remaining 2 gallons from the 3-gallon jug into the 4-gallon jug.

State Sequence:

(0,0) → (0,3) → (3,0) → (3,3) → (4,2) → (0,2) → (2,0)

Result: The 4-gallon jug contains exactly **2 gallons**.

Question 2 – Solution

- **Percepts:** Camera images, rock samples, soil samples, obstacles, temperature, battery level, position, atmospheric conditions.
- **Operating Environment:** Partially observable, dynamic, sequential, continuous, single-agent, uncertain.
- **Actions:** Move forward/backward, turn left/right, collect samples, capture images, analyze samples, avoid obstacles, transmit data, recharge battery.
- **Performance Measure:** Safe navigation, accurate sample collection, efficient energy usage, successful data transmission, mission completion.
- **Suitable Agent Architecture: Utility-Based Learning Agent**, as it can learn from experience and make optimal decisions in an uncertain environment.

Question 3 – Solution

The 8-Queens problem is solved using **Depth-First Search (DFS)** with **Backtracking**. Start with an empty 8 × 8 chessboard and place one queen in each column such that no two queens share the same row, column, or diagonal. If a conflict occurs, backtrack and try another position until all eight queens are safely placed.

One valid solution:

Column	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
Row	1	5	8	6	3	7	2	4

Question 4 – Solution

The suitable problem-solving agent is a **Goal-Based Agent** because its objective is to book a suitable cab and transport the customer safely to the destination.

Pseudocode:

```
START
Input Pickup Location
Input Destination
Display Available Cab Types
Select Cab Type
IF Cab Available THEN
   Assign Driver
   Confirm Booking
   Start Trip
ELSE
   Display "Cab Not Available"
ENDIF
END
```

Question 5 – Solution

Uniform Cost Search (UCS) expands the node with the **lowest cumulative path cost** until the goal node is reached.

Steps:

1. Start from node **S**.
2. Insert **S** into the priority queue.
3. Expand the node with the lowest cost.
4. Update the cumulative costs of neighboring nodes.
5. Repeat until **G** is reached.

Least-Cost Path:

S → B → C → G

Total Cost = 8

Thus, the UCS algorithm finds the minimum-cost path from the starting warehouse to the destination warehouse.